

Chapter 14. Block Device Drivers

This chapter deals with I/O drivers for block devices, i.e., for disks of every kind. The key aspect of a block device is the disparity between the time taken by the CPU and buses to read or write data and the speed of the disk hardware. Block devices have very high average access times. Each operation requires several milliseconds to complete, mainly because the disk controller must move the heads on the disk surface to reach the exact position where the data is recorded. However, when the heads are correctly placed, data transfer can be sustained at rates of tens of megabytes per second.

The organization of Linux block device handlers is quite involved. We won't be able to discuss in detail all the functions that are included in the block I/O subsystem of the kernel; however, we'll outline the general software architecture. As in the previous chapter, our objective is to explain how Linux supports the implementation of block device drivers, rather than showing how to implement one of them.

We start in the first section "[Block Devices Handling](#)" to explain the general architecture of the Linux block I/O subsystem. In the sections "[The Generic Block Layer](#)," "[The I/O Scheduler](#)," and "[Block Device Drivers](#)," we will describe the main components of the block I/O subsystem. Finally, in the last section, "[Opening a Block Device File](#)," we will outline the steps performed by the kernel when opening a block device file.

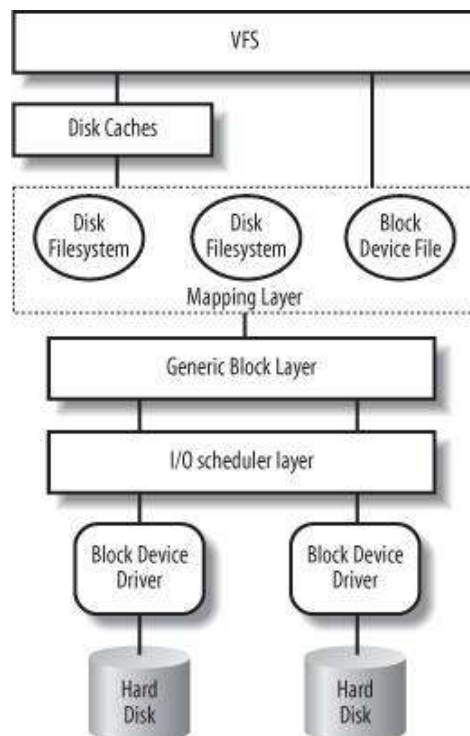
14.1. Block Devices Handling

Each operation on a block device driver involves a large number of kernel components; the most important ones are shown in [Figure 14-1](#).

Let us suppose, for instance, that a process issued a `read()` system call on some disk file we'll see that write requests are handled essentially in the same way. Here is what the kernel typically does to service the process request:

1. The service routine of the `read()` system call activates a suitable VFS function, passing to it a file descriptor and an offset inside the file. The Virtual Filesystem

Figure 14-1. Kernel components affected by a block device operation



is the upper layer of the block device handling architecture, and it provides a common file model adopted by all filesystems supported by Linux. We have described at length the VFS layer in [Chapter 12](#).

2. The VFS function determines if the requested data is already available and, if necessary, how to perform the read operation. Sometimes there is no need to access the data on disk, because the kernel keeps in RAM the data most recently read from or written to a block device. The disk cache mechanism is explained in [Chapter 15](#), while details on how the VFS handles the disk operations and how it interfaces with the disk cache and the filesystems are given in [Chapter 16](#).
3. Let's assume that the kernel must read the data from the block device, thus it must determine the physical location of that data. To do this, the kernel relies on the mapping layer, which typically executes two steps:
 - a. It determines the block size of the filesystem including the file and computes the extent of the

requested data in terms of file block numbers. Essentially, the file is seen as split in many blocks, and the kernel determines the numbers (indices relative to the beginning of file) of the blocks containing the requested data.

- b. Next, the mapping layer invokes a filesystem-specific function that accesses the file's disk inode and determines the position of the requested data on disk in terms of logical block numbers. Essentially, the disk is seen as split in blocks, and the kernel determines the numbers (indices relative to the beginning of the disk or partition) corresponding to the blocks storing the requested data. Because a file may be stored in nonadjacent blocks on disk, a data structure stored in the disk inode maps each file block number to a logical block number.^[*]

[*] However, if the read access was done on a raw block device file, the mapping layer does not invoke a filesystem-specific method; rather, it translates the offset in the block device file to a position inside the disk or disk partition corresponding to the device file.

We will see the mapping layer in action in [Chapter 16](#), while we will present some typical disk-based filesystems in [Chapter 18](#).

4. The kernel can now issue the read operation on the block device. It makes use of the generic block layer, which starts the I/O operations that transfer the requested data. In general, each I/O operation involves a group of blocks that are adjacent on disk. Because the requested data is not necessarily adjacent on disk, the generic block layer might start several I/O operations. Each I/O operation is represented by a "block I/O" (in short, "bio") structure, which collects all information needed by the lower components to satisfy the request.

The generic block layer hides the peculiarities of each hardware block device, thus offering an abstract view of the block devices. Because almost all block devices are disks, the generic block layer also provides some general data structures that describe "disks" and "disk partitions." We will discuss the generic block layer and the bio structure in the section "[The Generic Block Layer](#)" later in this chapter.

5. Below the generic block layer, the "I/O scheduler" sorts the pending I/O data transfer requests according to predefined kernel policies. The purpose of the scheduler is to group requests of data that lie near each other on the physical medium. We will describe this component in the section "[The I/O Scheduler](#)" later in this chapter.
6. Finally, the block device drivers take care of the actual data transfer by sending suitable commands to the hardware interfaces of the disk controllers. We will explain the overall organization of a generic block device driver in the section "[Block Device Drivers](#)" later in this chapter.

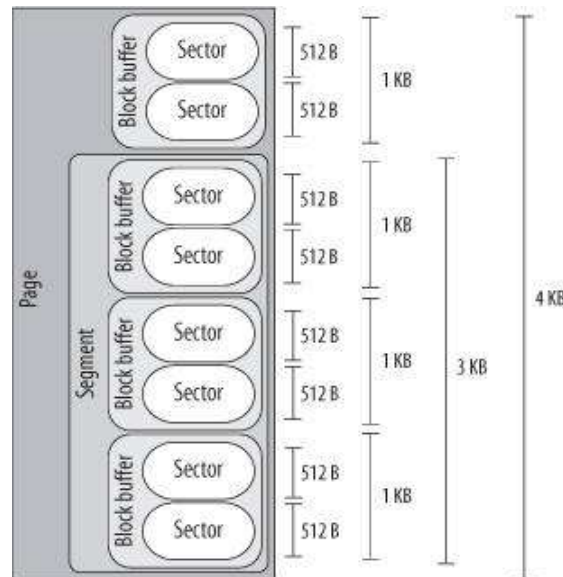
As you can see, there are many kernel components that are concerned with data stored in block devices; each of them manages the disk data using chunks of different length:

- The controllers of the hardware block devices transfer data in chunks of fixed length called "sectors." Therefore, the I/O scheduler and the block device drivers must manage sectors of data.
- The Virtual Filesystem, the mapping layer, and the filesystems group the disk data in logical units called "blocks." A block corresponds to the minimal disk storage unit inside a filesystem.
- As we will see shortly, block device drivers should be able to cope with "segments" of data: each segment is a memory page or a portion of a memory page including chunks of data that are physically adjacent on disk.
- The disk caches work on "pages" of disk data, each of which fits in a page frame.
- The generic block layer glues together all the upper and lower components, thus it knows about sectors, blocks, segments, and pages of data.

Even if there are many different chunks of data, they usually share the same physical RAM cells. For instance, [Figure 14-2](#) shows the layout of a 4,096-byte page. The upper kernel components see the page as composed of four block buffers of 1,024 bytes each. The last three blocks of the page are being transferred by the block device driver, thus they are inserted in a segment covering the last 3,072 bytes of the page. The hard disk

controller considers the segment as composed of six 512-byte sectors.

Figure 14-2. Typical layout of a page including disk data



In this chapter we describe the lower kernel components that handle the block devices generic block layer, I/O scheduler, and block device drivers thus we focus our attention on sectors, blocks, and segments.

14.1.1. Sectors

To achieve acceptable performance, hard disks and similar devices transfer several adjacent bytes at once. Each data transfer operation for a block device acts on a group of adjacent bytes called a sector. In the following discussion, we say that groups of bytes are adjacent when they are recorded on the disk surface in such a manner that a single seek operation can access them. Although the physical geometry of a disk is usually very complicated, the hard disk controller accepts commands that refer to the disk as a large array of sectors.

In most disk devices, the size of a sector is 512 bytes, although there are devices that use larger sectors (1,024 and 2,048 bytes). Notice that the sector should be considered as the basic unit of data transfer; it is never possible to transfer less than one sector, although most disk devices are capable of transferring several adjacent sectors at once.

In Linux, the size of a sector is conventionally set to 512 bytes; if a block device uses larger sectors, the corresponding low-level block device driver will do the necessary conversions. Thus, a group of data stored in a block device is identified on disk by its position the index of the first 512-byte sector and its length as number of 512-byte sectors. Sector indices are stored in 32- or 64-bit variables of type `sector_t`.

14.1.2. Blocks

While the sector is the basic unit of data transfer for the hardware devices, the block is the basic unit of data transfer for the VFS and, consequently, for the filesystems. For example, when the kernel accesses the contents of a file, it must first read from disk a block containing the disk inode of the file (see the section "[Inode Objects](#)" in [Chapter 12](#)). This block on disk corresponds to one or more adjacent sectors, which are looked at by the VFS as a single data unit.

In Linux, the block size must be a power of 2 and cannot be larger than a page frame. Moreover, it must be a multiple of the sector size, because each block must include an integral number of sectors. Therefore, on 80 x 86 architecture, the permitted block sizes are 512, 1,024, 2,048, and 4,096 bytes.

The block size is not specific to a block device. When creating a disk-based filesystem, the administrator may select the proper block size. Thus, several partitions on the same disk might make use of different block sizes. Furthermore, each read or write operation issued on a block device file is a "raw" access that bypasses the disk-based filesystem; the kernel executes it by using blocks of largest size (4,096 bytes).

Each block requires its own block buffer, which is a RAM memory area used by the kernel to store the block's content. When the kernel reads a block from disk, it fills the corresponding block buffer with the values obtained from the hardware device; similarly, when the kernel writes a block on disk, it updates the corresponding group of adjacent bytes on the hardware device with the actual values of the associated block buffer. The size of a block buffer always matches the size of the corresponding block.

Each buffer has a "buffer head" descriptor of type `buffer_head`. This descriptor contains all the information needed by the kernel to know how to handle the buffer; thus, before operating on each buffer, the kernel checks its buffer head. We will give a detailed explanation of all fields of the buffer head in [Chapter 15](#); in the present chapter, however, we will only consider a few fields: `b_page`, `b_data`, `b_blocknr`, and `b_bdev`.

The `b_page` field stores the page descriptor address of the page frame that includes the block buffer. If the page frame is in high memory, the `b_data` field stores the offset of the block buffer inside the page; otherwise, it stores the starting linear address of the block buffer itself. The `b_blocknr` field stores the logical block number (i.e., the index of the block inside the disk partition). Finally, the `b_bdev` field identifies the block device that is using the buffer head (see the section "[Block Devices](#)" later in this chapter).

14.1.3. Segments

We know that each disk I/O operation consists of transferring the contents of some adjacent sectors from some RAM locations. In almost all cases, the data transfer is directly performed by the disk controller with a DMA operation (see the section "[Direct Memory Access \(DMA\)](#)" in [Chapter 13](#)). The block device driver simply triggers the data transfer by sending suitable commands to the disk controller; once the data transfer is finished, the controller raises an interrupt to notify the block device driver.

The data transferred by a single DMA operation must belong to sectors that are adjacent on disk. This is a physical constraint: a disk controller that allows DMA transfers to non-adjacent sectors would have a poor transfer rate, because moving a read/write head on the disk surface is quite a slow operation.

Older disk controllers support "simple" DMA operations only: in each such operation, data is transferred from or to memory cells that are physically contiguous in RAM. Recent disk controllers, however, may also support the so-called scatter-gather DMA transfers: in each such operation, the data can be transferred from or to several noncontiguous memory areas.

For each scatter-gather DMA transfer, the block device driver must send to the disk controller:

- The initial disk sector number and the total number of sectors to be transferred

- A list of descriptors of memory areas, each of which consists of an address and a length.

The disk controller takes care of the whole data transfer; for instance, in a read operation the controller fetches the data from the adjacent disk sectors and scatters it into the various memory areas.

To make use of scatter-gather DMA operations, block device drivers must handle the data in units called segments. A segment is simply a memory page or a portion of a memory page that includes the data of some adjacent disk sectors. Thus, a scatter-gather DMA operation may involve several segments at once.

Notice that a block device driver does not need to know about blocks, block sizes, and block buffers. Thus, even if a segment is seen by the higher levels as a page composed of several block buffers, the block device driver does not care about it.

As we'll see, the generic block layer can merge different segments if the corresponding page frames happen to be contiguous in RAM and the corresponding chunks of disk data are adjacent on disk. The larger memory area resulting from this merge operation is called physical segment.

Yet another merge operation is allowed on architectures that handle the mapping between bus addresses and physical addresses through a dedicated bus circuitry (the IO-MMU; see the section "[Direct Memory Access \(DMA\)](#)" in [Chapter 13](#)). The memory area resulting from this kind of merge operation is called hardware segment. Because we will focus on the 80 x 86 architecture, which has no such dynamic mapping between bus addresses and physical addresses, we will assume in the rest of this chapter that hardware segments always coincide with physical segments.

14.2. The Generic Block Layer

The generic block layer is a kernel component that handles the requests for all block devices in the system. Thanks to its functions, the kernel may easily:

- Put data buffers in high memorythe page frame(s) will be mapped in the kernel linear address space only when the CPU must access the data, and will be unmapped right after.
- Implementwith some additional efforta "zero-copy" schema, where disk data is directly put in the User Mode address space without being copied to kernel memory first; essentially, the buffer used by the kernel for the I/O transfer lies in a page frame mapped in the User Mode linear address space of a process.
- Manage logical volumessuch as those used by LVM (the Logical Volume Manager) and RAID (Redundant Array of Inexpensive Disks): several disk partitions, even on different block devices, can be seen as a single partition.
- Exploit the advanced features of the most recent disk controllers, such as large onboard disk caches , enhanced DMA capabilities, onboard scheduling of the I/O transfer requests, and so on.

14.2.1. The Bio Structure

The core data structure of the generic block layer is a descriptor of an ongoing I/O block device operation called bio. Each bio essentially includes an identifier for a disk storage areathe initial sector number and the number of sectors included in the storage areaand one or more segments describing the memory areas involved in the I/O operation. A bio is implemented by the `bio` data structure, whose fields are listed in [Table 14-1](#).

Table 14-1. The fields of the bio structure

Type	Field	Description
sector_t	bi_sector	First sector on disk of block I/O operation
struct bio *	bi_next	Link to the next bio in the request queue
struct	bi_bdev	Pointer to block device descriptor
block_device *	bi_flags	Bio status flags
unsigned long	bi_rw	I/O operation flags
unsigned long	bi_vcnt	Number of segments in the bio's <code>bio_vec</code> array
unsigned short	bi_idx	Current index in the bio's <code>bio_vec</code> array of segments
unsigned short	bi_phys_segments	Number of physical segments of the bio after merging
unsigned short	bi_hw_segments	Number of hardware segments after merging
unsigned int	bi_size	Bytes (yet) to be transferred
unsigned int	bi_hw_front_size	Used by the hardware segment merge algorithm
unsigned int	bi_hw_back_size	Used by the hardware segment merge algorithm
unsigned int	bi_max_vecs	Maximum allowed number of segments in the bio's <code>bio_vec</code> array
struct bio_vec *	bi_io_vec	Pointer to the bio's <code>bio_vec</code> array of segments
bio_end_io_t *	bi_end_io	Method invoked at the end of bio's I/O operation
atomic_t	bi_cnt	Reference counter for the bio
void *	bi_private	Pointer used by the generic block layer and the I/O completion method of the block device driver
bio_destructor_t *	bi_destructor	Destructor method (usually <code>bio_destructor()</code>) invoked when the bio is being freed

Each segment in a bio is represented by a `bio_vec` data structure, whose fields are listed in [Table 14-2](#). The `bi_io_vec` field of the bio points to the first element of an array of `bio_vec` data structures, while the `bi_vcnt` field stores the current number of elements in the array.

Table 14-2. The fields of the bio_vec structure

Type

Field

Description

```
struct page *
```

```
bv_page
```

Pointer to the page descriptor of the segment's page frame

```
unsigned int
```

```
bv_len
```

Length of the segment in bytes

```
unsigned int
```

```
bv_offset
```

Offset of the segment's data in the page frame

The contents of a `bio` descriptor keep changing during the block I/O operation. For instance, if the block device driver cannot perform the whole data transfer with one scatter-gather DMA operation, the `bi_idx` field is updated to keep track of the first segment in the bio that is yet to be transferred. To iterate over the segments of a bio starting from the current segment at index `bi_idx` a device driver can execute the `bio_for_each_segment` macro.

When the generic block layer starts a new I/O operation, it allocates a new bio structure by invoking the `bio_alloc( )` function. Usually, bios are allocated through the slab allocator, but the kernel also keeps a small memory pool of bios to be used when memory is scarce (see the section "[Memory Pools](#)" in [Chapter 8](#)). The kernel also keeps a memory pool for the `bio_vec` structures; after all, it would not make sense to allocate a bio without being able to allocate the segment descriptors to be included in the bio. Correspondingly, the `bio_put( )` function decrements the reference counter (`bi_cnt`) of a bio and, if the counter becomes zero, it releases the bio structure and the related `bio_vec` structures.

14.2.2. Representing Disks and Disk Partitions

A disk is a logical block device that is handled by the generic block layer. Usually a disk corresponds to a hardware block device such as a hard disk, a floppy disk, or a CD-ROM disk. However, a disk can be a virtual device built upon several physical disk partitions, or a storage area living in some dedicated pages of RAM. In any case, the upper kernel components operate on all disks in the same way thanks to the services offered by the generic block layer.

A disk is represented by the `gendisk` object, whose fields are shown in [Table 14-3](#).

Table 14-3. The fields of the `gendisk` object

Type

Field

Description

4

int

major

Major number of the disk

int

first_minor

First minor number associated with the disk

int

minors

Range of minor numbers associated with the disk

char [32]

disk_name

Conventional name of the disk (usually, the canonical name of the corresponding device file)

struct hd_struct **

part

Array of partition descriptors for the disk

struct

block_device_operations *

fops

Pointer to a table of block device methods

struct request_queue *

queue

Pointer to the request queue of the disk (see "[Request Queue Descriptors](#)" later in this chapter)

void *

private_data

Private data of the block device driver

sector_t

capacity

Size of the storage area of the disk (in number of sectors)

`int`

`flags`

Flags describing the kind of disk (see below)

`char [64]`

`devfs_name`

Device filename in the (nowadays deprecated) devfs special filesystem

`int`

`number`

No longer used

`struct device *`

`driverfs_dev`

Pointer to the `device` object of the disk's hardware device (see the section "[Components of the Device Driver Model](#)" in [Chapter 13](#))

`struct kobject`

`kobj`

Embedded kobject (see the section "[Kobjects](#)" in [Chapter 13](#))

`struct timer_rand_state *`

`random`

Pointer to a data structure that records the timing of the disk's interrupts; used by the kernel built-in random number generator

`int`

`policy`

Set to 1 if the disk is read-only (write operations forbidden), 0 otherwise

`atomic_t`

`sync_io`

Counter of sectors written to disk, used only for RAID

`unsigned long`

stamp

Timestamp used to determine disk queue usage statistics

unsigned long

stamp_idle

Same as above

int

in_flight

Number of ongoing I/O operations

struct disk_stats *

dkstats

Statistics about per-CPU disk usage

The `flags` field stores information about the disk. The most important flag is `GENHD_FL_UP`: if it is set, the disk is initialized and working. Another relevant flag is `GENHD_FL_REMOVABLE`, which is set if the disk is a removable support, such as a floppy disk or a CD-ROM.

The `fops` field of the `gendisk` object points to a `block_device_operations` table, which stores a few custom methods for crucial operations of the block device (see [Table 14-4](#)).

Table 14-4. The methods of the block devices

Method

Triggers

open

Opening the block device file

release

Closing the last reference to a block device file

ioctl

Issuing an `ioctl( )` system call on the block device file (uses the big kernel lock)

compat_ioctl

Issuing an `ioctl( )` system call on the block device file (does not use the big kernel lock)

media_changed

Checking whether the removable media has been changed (e.g., floppy disk)

`revalidate_disk`

Checking whether the block device holds valid data

Hard disks are commonly split into logical partitions . Each block device file may represent either a whole disk or a partition inside the disk. For instance, a master EIDE disk might be represented by a device file `/dev/hda` having major number 3 and minor number 0; the first two partitions inside the disk might be represented by device files `/dev/hda1` and `/dev/hda2` having major number 3 and minor numbers 1 and 2, respectively. In general, the partitions inside a disk are characterized by consecutive minor numbers.

If a disk is split in partitions, their layout is kept in an array of `hd_struct` structures whose address is stored in the `part` field of the `gendisk` object. The array is indexed by the relative index of the partition inside the disk. The fields of the `hd_struct` descriptor are listed in [Table 14-5](#).

Table 14-5. The fields of the `hd_struct` descriptor

Type

Field

Description

`sector_t`

`start_sect`

Starting sector of the partition inside the disk

`sector_t`

`nr_sects`

Length of the partition (number of sectors)

`struct kobject`

`kobj`

Embedded kobject (see the section "[Kobjects](#)" in [Chapter 13](#))

`unsigned int`

`reads`

Number of read operations issued on the partition

`unsigned int`

`read_sectors`

Number of sectors read from the partition

`unsigned int`

`writes`

Number of write operations issued on the partition

`unsigned int`

`write_sectors`

Number of sectors written into the partition

`int`

`policy`

Set to 1 if the partition is read-only, 0 otherwise

`int`

`partno`

The relative index of the partition inside the disk

When the kernel discovers a new disk in the system (in the boot phase, or when a removable media is inserted in a drive, or when an external disk is attached at run-time), it invokes the `alloc_disk( )` function, which allocates and initializes a new `gendisk` object and, if the new disk is split in several partitions, a suitable array of `hd_struct` descriptors. Then, it invokes the `add_disk( )` function to insert the new `gendisk` descriptor into the data structures of the generic block layer (see the section "[Device Driver Registration and Initialization](#)" later in this chapter).

14.2.3. Submitting a Request

Let us describe the common sequence of steps executed by the kernel when submitting an I/O operation request to the generic block layer. We'll assume that the requested chunks of data are adjacent on disk and that the kernel has already determined their physical location.

The first step consists in executing the `bio_alloc( )` function to allocate a new bio descriptor. Then, the kernel initializes the bio descriptor by setting a few fields:

- The `bi_sector` field is set to the initial sector number of the data (if the block device is split in several partitions, the sector number is relative to the start of the partition).
- The `bi_size` field is set to the number of sectors covering the data.
- The `bi_bdev` field is set to the address of the block device descriptor (see the section "[Block Devices](#)" later in this chapter).
- The `bi_io_vec` field is set to the initial address of an array of `bio_vec` data structures, each of which describes a segment (memory buffer) involved in the I/O operation; moreover, the `bi_vcnt`

field is set to the total number of segments in the bio.

- The `bi_rw` field is set with the flags of the requested operation. The most important flag specifies the data transfer direction: `READ (0)` or `WRITE (1)`.
- The `bi_end_io` field is set to the address of a completion procedure that is executed whenever the I/O operation on the bio is completed.

Once the `bio` descriptor has been properly initialized, the kernel invokes the `generic_make_request()` function, which is the main entry point of the generic block layer. The function essentially executes the following steps:

1. Checks that `bio->bi_sector` does not exceed the number of sectors of the block device. If it does, the function sets the `BIO_EOF` flag of `bio->bi_flags`, prints a kernel error message, invokes the `bio_endio()` function, and terminates. `bio_endio()` updates the `bi_size` and `bi_sector` fields of the bio descriptor, and it invokes the `bi_end_io` bio's method. The implementation of the latter function essentially depends on the kernel component that has triggered the I/O data transfer; we will see some examples of `bi_end_io` methods in the following chapters.
2. Gets the request queue `q` associated with the block device (see the section "[Request Queue Descriptors](#)" later in this chapter); its address can be found in the `bd_disk` field of the block device descriptor, which in turn is pointed to by the `bio->bi_bdev` field.
3. Invokes `block_wait_queue_running()` to check whether the I/O scheduler currently in use is being dynamically replaced; in this case, the function puts the process to sleep until the new I/O scheduler is started (see the next section "[The I/O Scheduler](#)").
4. Invokes `blk_partition_remap()` to check whether the block device refers to a disk partition (`bio->bi_bdev` not equal to `bio->bi_dev->bd_contains`; see the section "[Block Devices](#)" later in this chapter). In this case, the function gets the `hd_struct` descriptor of the partition from the `bio->bi_bdev` field to perform the following substeps:
 - a. Updates the `read_sectors` and `reads` fields, or the `write_sectors` and `writes` fields, of the `hd_struct` descriptor, according to the direction of data transfer.
 - b. Adjusts the `bio->bi_sector` field so as to transform the sector number relative to the start of the partition to a sector number relative to the whole disk.
 - c. Sets the `bio->bi_bdev` field to the block device descriptor of the whole disk (`bio->bd_contains`).

From now on, the generic block layer, the I/O scheduler, and the device driver forget about disk partitioning, and work directly with the whole disk.
5. Invokes the `q->make_request_fn` method to insert the `bio` request in the request queue `q`.
6. Returns.

We will discuss a typical implementation of the `make_request_fn` method in the section "[Issuing a Request to the I/O Scheduler](#)" later in this chapter.

14.3. The I/O Scheduler

Although block device drivers are able to transfer a single sector at a time, the block I/O layer does not perform an individual I/O operation for each sector to be accessed on disk; this would lead to poor disk performance, because locating the physical position of a sector on the disk surface is quite time-consuming. Instead, the kernel tries, whenever possible, to cluster several sectors and handle them as a whole, thus reducing the average number of head movements.

When a kernel component wishes to read or write some disk data, it actually creates a block device request. That request essentially describes the requested sectors and the kind of operation to be performed on them (read or write). However, the kernel does not satisfy a request as soon as it is created; the I/O operation is just scheduled and will be performed at a later time. This artificial delay is paradoxically the crucial mechanism for boosting the performance of block devices. When a new block data transfer is requested, the kernel checks whether it can be satisfied by slightly enlarging a previous request that is still waiting (i.e., whether the new request can be satisfied without further seek operations). Because disks tend to be accessed sequentially, this simple mechanism is very effective.

Deferring requests complicates block device handling. For instance, suppose a process opens a regular file and, consequently, a filesystem driver wants to read the corresponding inode from disk. The block device driver puts the request on a queue, and the process is suspended until the block storing the inode is transferred. However, the block device driver itself cannot be blocked, because any other process trying to access the same disk would be blocked as well.

To keep the block device driver from being suspended, each I/O operation is processed asynchronously. In particular, block device drivers are interrupt-driven (see the section "[Monitoring I/O Operations](#)" in the previous chapter): the generic block layer invokes the I/O scheduler to create a new block device request or to enlarge an already existing one and then terminates. The block device driver, which is activated at a later time, invokes the strategy routine to select a pending request and satisfy it by issuing suitable commands to the disk controller. When the I/O operation terminates, the disk controller raises an interrupt and the corresponding handler invokes the strategy routine again, if necessary, to process another pending request.

Each block device driver maintains its own request queue, which contains the list of pending requests for the device. If the disk controller is handling several disks, there is usually one request queue for each physical block device. I/O scheduling is performed separately on each request queue, thus increasing disk performance.

14.3.1. Request Queue Descriptors

Each request queue is represented by means of a large `request_queue` data structure whose fields are listed in [Table 14-6](#).

Table 14-6. The fields of the request queue descriptor

Type	Field	Description
<code>struct list_head</code>	<code>queue_head</code>	List of pending requests Pointer to descriptor of the request in the queue to be considered first for possible merging
<code>struct request *</code>	<code>last_merge</code>	Pointer to the elevator object (see the later section " I/O Scheduling Algorithms ")
<code>elevator_t *</code>	<code>elevator</code>	Data structure used for allocation of request descriptors
<code>struct request_list</code>	<code>rq</code>	Method that implements the entry point of the strategy routine of the driver
<code>request_fn_proc *</code>	<code>request_fn</code>	Method to check whether it is possible to merge a bio to the last request in the queue
<code>merge_request_fn *</code>	<code>back_merge_fn</code>	Method to check whether it is possible to merge a bio to the first request in the queue
<code>merge_request_fn *</code>	<code>front_merge_fn</code>	Method to attempt merging two adjacent requests in the queue
<code>merge_requests_fn *</code>	<code>merge_requests_fn</code>	Method invoked when a new request has to be inserted in the queue
<code>make_request_fn *</code>	<code>make_request_fn</code>	

<code>prep_rq_fn *</code>	<code>prep_rq_fn</code>	Method to build the commands to be sent to the hardware device to process this request
<code>unplug_fn *</code>	<code>unplug_fn</code>	Method to unplug the block device (see the section " Activating the Block Device Driver " later in the chapter)
<code>merge_bvec_fn *</code>	<code>merge_bvec_fn</code>	Method that returns the number of bytes that can be inserted into an existing bio when adding a new segment (usually undefined)
<code>activity_fn *</code>	<code>activity_fn</code>	Method invoked when a request is added to a queue (usually undefined)
<code>issue_flush_fn *</code>	<code>issue_flush_fn</code>	Method invoked when a request queue is flushed (the queue is emptied by processing all requests in a row)
<code>struct timer_list</code>	<code>unplug_timer</code>	Dynamic timer used to perform device unplugging (see the later section " Activating the Block Device Driver ")
<code>int</code>	<code>unplug_thresh</code>	If the number of pending requests in the queue exceeds this value, the device is immediately unplugged (default is 4)
<code>unsigned long</code>	<code>unplug_delay</code>	Time delay before device unplugging (default is 3 milliseconds)
<code>struct work_struct</code>	<code>unplug_work</code>	Work queue used to unplug the device (see the later section " Activating the Block Device Driver ")
<code>struct backing_dev_info</code>	<code>backing_dev_info</code>	See the text following this table
<code>void *</code>	<code>queuedata</code>	Pointer to private data of the block device driver
<code>void *</code>	<code>activity_data</code>	Private data used by the <code>activity_fn</code>

		method
unsigned long	bounce_pfn	Page frame number above which buffer bouncing must be used (see the section " Submitting a Request " later in this chapter)
int	bounce_gfp	Memory allocation flags for bounce buffers
unsigned long	queue_flags	Set of flags describing the queue status
spinlock_t *	queue_lock	Pointer to request queue lock
struct kobject	kobj	Embedded kobject for the request queue
unsigned long	nr_requests	Maximum number of requests in the queue
unsigned int	nr_congestion_on	Queue is considered congested if the number of pending requests rises above this threshold
unsigned int	nr_congestion_off	Queue is considered not congested if the number of pending requests falls below this threshold
unsigned int	nr_batching	Maximum number (usually 32) of pending requests that can be submitted even when the queue is full by a special "batcher" process
unsigned short	max_sectors	Maximum number of sectors handled by a single request (tunable)
unsigned short	max_hw_sectors	Maximum number of sectors handled by a single request (hardware constraint)
unsigned short	max_phys_segments	Maximum number of physical segments handled by a single request
unsigned short	max_hw_segments	Maximum number of hardware segments handled by a single request (the maximum number of distinct memory areas in a

		scatter-gather DMA operation)
unsigned short	hardsect_size	Size in bytes of a sector
unsigned int	max_segment_size	Maximum size of a physical segment (in bytes)
unsigned long	seg_boundary_mask	Memory boundary mask for segment merging
unsigned int	dma_alignment	Alignment bitmap for initial address and length of DMA buffers (default 511)
struct	queue_tags	Bitmap of free/busy tags (used for tagged requests)
blk_queue_tag *	refcnt	Reference counter of the queue
atomic_t	in_flight	Number of pending requests in the queue
unsigned int	sg_timeout	User-defined command time-out (used only by SCSI generic devices)
unsigned int	sg_reserved_size	Essentially unused
struct list_head	drain_list	Head of a list of requests temporarily delayed until the I/O scheduler is dynamically replaced

Essentially, a request queue is a doubly linked list whose elements are request descriptors (that is, request data structures; see the next section). The `queue_head` field of the request queue descriptor stores the head (the first dummy element) of the list, while the pointers in the `queuelist` field of the request descriptor link each request to the previous and next elements in the list. The ordering of the elements in the queue list is specific to each block device driver; the I/O scheduler offers, however, several predefined ways of ordering elements, which are discussed in the later section "[The I/O Scheduler](#)."

The `backing_dev_info` field is a small object of type `backing_dev_info`, which stores information about the I/O data flow traffic for the underlying hardware block device. For instance, it holds information about read-ahead and about request queue congestion state.

14.3.2. Request Descriptors

Each pending request for a block device is represented by a request descriptor, which is stored in the request data structure illustrated in [Table 14-7](#).

Table 14-7. The fields of the request descriptor

Type

Field

Description

`struct list_head``queuelist`

Pointers for request queue list

`unsigned long``flags`

Flags of the request (see below)

`sector_t``sector`

Number of the next sector to be transferred

`unsigned long``nr_sectors`

Number of sectors yet to be transferred in the whole request

`unsigned int``current_nr_sectors`

Number of sectors in the current segment of the current bio yet to be transferred

`sector_t``hard_sector`

Number of the next sector to be transferred

`unsigned long``hard_nr_sectors`

Number of sectors yet to be transferred in the whole request (updated by the generic block layer)

`unsigned int``hard_cur_sectors`

Number of sectors in the current segment of the current bio yet to be transferred (updated by the generic block layer)

```
struct bio *
```

```
bio
```

First bio in the request that has not been completely transferred

```
struct bio *
```

```
biotail
```

Last bio in the request list

```
void *
```

```
elevator_private
```

Pointer to private data for the I/O scheduler

```
int
```

```
rq_status
```

Request status: essentially, either RQ_ACTIVE or RQ_INACTIVE

```
struct gendisk *
```

```
rq_disk
```

The descriptor of the disk referenced by the request

```
int
```

```
errors
```

Counter for the number of I/O errors that occurred on the current transfer

```
unsigned long
```

```
start_time
```

Request's starting time (in jiffies)

```
unsigned short
```

```
nr_phys_segments
```

Number of physical segments of the request

```
unsigned short
```

```
nr_hw_segments
```

Number of hardware segments of the request

int

tag

Tag associated with the request (only for hardware devices supporting multiple outstanding data transfers)

char *

buffer

Pointer to the memory buffer of the current data transfer (NULL if the buffer is in high-memory)

int

ref_count

Reference counter for the request

request_queue_t *

q

Pointer to the descriptor of the request queue containing the request

struct request_list *

rl

Pointer to `request_list` data structure

struct completion *

waiting

Completion for waiting for the end of the data transfers (see the section "[Completions](#)" in [Chapter 5](#))

void *

special

Pointer to data used when the request includes a "special" command to the hardware device

unsigned int

cmd_len

Length of the commands in the `cmd` field

unsigned char []

cmd

Buffer containing the pre-built commands prepared by the request queue's `prep_rq_fn` method

unsigned int

`data_len`

Usually, the length of data in the buffer pointed to by the `data` field

void *

`data`

Pointer used by the device driver to keep track of the data to be transferred

unsigned int

`sense_len`

Length of buffer pointed to by the `sense` field (0 if the sense field is NULL)

void *

`sense`

Pointer to buffer used for output of sense commands

unsigned int

`timeout`

Request's time-out

struct

`request_pm_state *`

`pm`

Pointer to a data structure used for power-management commands

Each request consists of one or more bio structures. Initially, the generic block layer creates a request including just one bio. Later, the I/O scheduler may "extend" the request either by adding a new segment to the original bio, or by linking another bio structure into the request. This is possible when the new data is physically adjacent to the data already in the request. The `bio` field of the request descriptor points to the first bio structure in the request, while the `biotail` field points to the last bio. The `rq_for_each_bio` macro implements a loop that iterates over all bios included in a request.

Several fields of the request descriptor may dynamically change. For instance, as soon as the chunks of data referenced in a bio have all been transferred, the `bio` field is updated so that it points to the next bio in the request list. Meanwhile, new bios can be added to the tail of the request list, so the `biotail` field may also change.

Several other fields of the request descriptor are modified either by the I/O scheduler or the device driver while the disk sectors are being transferred. For instance, the `nr_sectors` field stores the number of sectors yet to be transferred in the whole request, while the `current_nr_sectors` field stores the number of sectors yet to be transferred in the current bio.

The `flags` field stores a large number of flags, which are listed in [Table 14-8](#). The most important one is, by far, `REQ_RW`, which determines the direction of the data transfer.

Table 14-8. The flags of the request descriptor

Flag

Description

`REQ_RW`

Direction of data transfer: READ (0) or WRITE (1)

`REQ_FAILFAST`

Requests says to not retry the I/O operation in case of error

`REQ_SOFTBARRIER`

Request acts as a barrier for the I/O scheduler

`REQ_HARDBARRIER`

Request acts as a barrier for the I/O scheduler and the device driver it should be processed after older requests and before newer ones

`REQ_CMD`

Request includes a normal read or write I/O data transfer

`REQ_NOMERGE`

Request should not be extended or merged with other requests

`REQ_STARTED`

Request is being processed

`REQ_DONTPREP`

Do not invoke the `prep_rq_fn` request queue's method to prepare in advance the commands to be sent to the hardware device

`REQ_QUEUED`

Request is tagged that is, it refers to a hardware device that can manage many outstanding data transfers at the same time

`REQ_PC`

Request includes a direct command to be sent to the hardware device

REQ_BLOCK_PC

Same as previous flag, but the command is included in a bio

REQ_SENSE

Request includes a "sense" request command (for SCSI and ATAPI devices)

REQ_FAILED

Set when a sense or direct command in the request did not work as expected

REQ_QUIET

Request says to not generate kernel messages in case of I/O errors

REQ_SPECIAL

Request includes a special command for the hardware device (e.g., drive reset)

REQ_DRIVE_CMD

Request includes a special command for IDE disks

REQ_DRIVE_TASK

Request includes a special command for IDE disks

REQ_DRIVE_TASKFILE

Request includes a special command for IDE disks

REQ_PREEMPT

Request replaces the current request in front of the queue (only for IDE disks)

REQ_PM_SUSPEND

Request includes a power-management command to suspend the hardware device

REQ_PM_RESUME

Request includes a power-management command to awaken the hardware device

REQ_PM_SHUTDOWN

Request includes a power-management command to switch off the hardware device

REQ_BAR_PREFLUSH

Request includes a "flush queue" command to be sent to the disk controller

REQ_BAR_POSTFLUSH

Request includes a "flush queue" command, which has been sent to the disk controller

14.3.2.1. Managing the allocation of request descriptors

The limited amount of free dynamic memory may become, under very heavy loads and high disk activity, a bottleneck for processes that want to add a new request into a request queue `q`. To cope with this kind of situation, each `request_queue` descriptor includes a `request_list` data structure, which consists of:

- A pointer to a memory pool of request descriptors (see the section "[Memory Pools](#)" in [Chapter 8](#)).
- Two counters for the number of requests descriptors allocated for READ and WRITE requests, respectively.
- Two flags indicating whether a recent allocation for a READ or WRITE request, respectively, failed.
- Two wait queues storing the processes sleeping for available READ and WRITE request descriptors, respectively.
- A wait queue for the processes waiting for a request queue to be flushed (emptied).

The `blk_get_request( )` function tries to get a free request descriptor from the memory pool of a given request queue; if memory is scarce and the memory pool is exhausted, the function either puts the current process to sleep or if the kernel control path cannot block returns NULL. If the allocation succeeds, the function stores in the `rl` field of the request descriptor the address of the `request_list` data structure of the request queue. The `blk_put_request( )` function releases a request descriptor; if its reference counter becomes zero, the descriptor is given back to the memory pool from which it was taken.

14.3.2.2. Avoiding request queue congestion

Each request queue has a maximum number of allowed pending requests. The `nr_requests` field of the request descriptor stores the maximum number of allowed pending requests for each data transfer direction. By default, a queue has at most 128 pending read requests and 128 pending write requests. If the number of pending read (write) requests exceeds `nr_requests`, the queue is marked as full by setting the `QUEUE_FLAG_READFULL` (`QUEUE_FLAG_WRITEFULL`) flag in the `queue_flags` field of the request queue descriptor, and blockable processes trying to add requests for that data transfer direction are put to sleep in the corresponding wait queue of the `request_list` data structure.

A filled-up request queue impacts negatively on the system's performance, because it forces many processes to sleep while waiting for the completion of I/O data transfers. Thus, if the number of pending requests for a given direction exceeds the value stored in the `nr_congestion_on` field of the request descriptor (by default, 113), the kernel regards the queue as congested and tries to slow down the creation rate of the new requests. A congested request queue becomes uncongested when the number of pending requests falls below the value of the `nr_congestion_off` field (by default, 111). The `blk_congestion_wait( )` function puts the current process to sleep until any request queue becomes uncongested or a time-out elapses.

14.3.3. Activating the Block Device Driver

As we saw earlier, it's expedient to delay activation of the block device driver in order to increase the chances of clustering requests for adjacent blocks. The delay is accomplished through a technique known as device plugging and unplugging.^[*] As long as a block device driver is plugged, the device driver is not activated even if there are requests to be processed in the driver's queues.

[*] If you are confused by the terms "plugging" and "unplugging," you might consider them equivalent to "de-activating" and "activating," respectively.

The `blk_plug_device( )` function plugs a block device or more precisely, a request queue serviced by some block device driver. Essentially, the function receives as an argument the address `q` of a request queue descriptor. It sets the `QUEUE_FLAG_PLUGGED` bit in the `q->queue_flags` field; then, it restarts the dynamic timer embedded in the `q->unplug_timer` field.

The `blk_remove_plug( )` function unplugs a request queue `q`: it clears the `QUEUE_FLAG_PLUGGED` flag and cancels the execution of the `q->unplug_timer` dynamic timer. This function can be explicitly invoked by the kernel when all mergeable requests "in sight" have been added to the queue. Moreover, the I/O scheduler unplugs a request queue if the number of pending requests in the queue exceeds the value stored in the `unplug_thres` field of the request queue descriptor (by default, 4).

If a device remains plugged for a time interval of length `q->unplug_delay` (usually 3 milliseconds), the dynamic timer activated by `blk_plug_device( )` elapses, thus the `blk_unplug_timeout( )` function is executed. As a consequence, the `kblockd` kernel thread servicing the `kblockd_workqueue` work queue is awakened (see the section "[Work Queues](#)" in [Chapter 4](#)). This kernel thread executes the function whose address is stored in the `q->unplug_work` data structure that is, the `blk_unplug_work( )` function. In turn, this function invokes the `q->unplug_fn` method of the request queue, which is usually implemented by the `generic_unplug_device( )` function. The `generic_unplug_device( )` function takes care of unplugging the block device: first, it checks whether the queue is still active; then, it invokes `blk_remove_plug( )`; and finally, it executes the strategy routine `request_fn` method to start processing the next request in the queue (see the section "[Device Driver Registration and Initialization](#)" later in this chapter).

14.3.4. I/O Scheduling Algorithms

When a new request is added to a request queue, the generic block layer invokes the I/O scheduler to determine that exact position of the new element in the queue. The I/O scheduler tries to keep the request queue sorted sector by sector. If the requests to be processed are taken sequentially from the list, the amount of disk seeking is significantly reduced because the disk head moves in a linear way from the inner track to the outer one (or vice versa) instead of jumping randomly from one track to another. This heuristic is reminiscent of the algorithm used by elevators when dealing with requests coming from different floors to go up or down. The elevator moves in one direction; when the last booked floor is reached in one direction, the elevator changes direction and starts moving in the other direction. For this reason, I/O schedulers are also called elevators.

Under heavy load, an I/O scheduling algorithm that strictly follows the order of the sector numbers is not going to work well. In this case, indeed, the completion time of a data transfer strongly depends on the physical position of the data on the disk. Thus, if a device driver is processing requests near the top of the queue (lower sector numbers), and new requests with low sector numbers are continuously added to the queue, then the requests in the tail of the queue can easily starve. I/O scheduling algorithms are thus quite sophisticated.

Currently, Linux 2.6 offers four different types of I/O schedulers or elevators called "Anticipatory,"

"Deadline," "CFQ (Complete Fairness Queueing)," and "Noop (No Operation)." The default elevator used by the kernel for most block devices is specified at boot time with the kernel parameter `elevator=<name>`, where `<name>` is one of the following: `as`, `deadline`, `cfq`, and `noop`. If no boot time argument is given, the kernel uses the "Anticipatory" I/O scheduler. Anyway, a device driver can replace the default elevator with another one; a device driver can also define its custom I/O scheduling algorithm, but this is very seldom done.

Furthermore, the system administrator can change at runtime the I/O scheduler for a specific block device. For instance, to change the I/O scheduler used in the master disk of the first IDE channel, the administrator can write an elevator name into the `/sys/block/hda/queue/scheduler` file of the `sysfs` special filesystem (see the section "[The sysfs Filesystem](#)" in [Chapter 13](#)).

The I/O scheduler algorithm used in a request queue is represented by an elevator object of type `elevator_t`; its address is stored in the `elevator` field of the request queue descriptor. The elevator object includes several methods covering all possible operations of the elevator: linking and unlinking the elevator to a request queue, adding and merging requests to the queue, removing requests from the queue, getting the next request to be processed from the queue, and so on. The elevator object also stores the address of a table including all information required to handle the request queue. Furthermore, each request descriptor includes an `elevator_private` field that points to an additional data structure used by the I/O scheduler to handle the request.

Let us now briefly describe the four I/O scheduling algorithms, from the simplest one to the most sophisticated one. Be warned that designing an I/O scheduler is much like designing a CPU scheduler (see [Chapter 7](#)): the heuristics and the values of the adopted constants are the result of an extensive amount of testing and benchmarking.

Generally speaking, all algorithms make use of a dispatch queue, which includes all requests sorted according to the order in which the requests should be processed by the device driver. The next request to be serviced by the device driver is always the first element in the dispatch queue. The dispatch queue is actually the request queue rooted at the `queue_head` field of the request queue descriptor. Almost all algorithms also make use of additional queues to classify and sort requests. All of them allow the device driver to add bios to existing requests and, if necessary, to merge two "adjacent" requests.

14.3.4.1. The "Noop" elevator

This is the simplest I/O scheduling algorithm. There is no ordered queue: new requests are always added either at the front or at the tail of the dispatch queue, and the next request to be processed is always the first request in the queue.

14.3.4.2. The "CFQ" elevator

The main goal of the "Complete Fairness Queueing" elevator is ensuring a fair allocation of the disk I/O bandwidth among all the processes that trigger the I/O requests. To achieve this result, the elevator makes use of a large number of sorted queues by default, 64 that store the requests coming from the different processes. Whenever a request is handed to the elevator, the kernel invokes a hash function that converts the thread group identifier of the current process (usually it corresponds to the PID, see the section "[Identifying a Process](#)" in [Chapter 3](#)) into the index of a queue; then, the elevator inserts the new request at the tail of this queue. Therefore, requests coming from the same process are always inserted in the same queue.

To refill the dispatch queue, the elevator essentially scans the I/O input queues in a round-robin fashion, selects the first nonempty queue, and moves a batch of requests from that queue into the tail of the dispatch

queue.

14.3.4.3. The "Deadline" elevator

Besides the dispatch queue, the "Deadline" elevator makes use of four queues. Two of them the sorted queues include the read and write requests, respectively, ordered according to their initial sector numbers. The other two the deadline queues include the same read and write requests sorted according to their "deadlines." These queues are introduced to avoid request starvation, which occurs when the elevator policy ignores for a very long time a request because it prefers to handle other requests that are closer to the last served one. A request deadline is essentially an expire timer that starts ticking when the request is passed to the elevator. By default, the expire time of read requests is 500 milliseconds, while the expire time for write requests is 5 seconds. Read requests are privileged over write requests because they usually block the processes that issued them. The deadline ensures that the scheduler looks at a request if it's been waiting a long time, even if it is low in the sort.

When the elevator must replenish the dispatch queue, it first determines the data direction of the next request. If there are both read and write requests to be dispatched, the elevator chooses the "read" direction, unless the "write" direction has been discarded too many times (to avoid write requests starvation).

Next, the elevator checks the deadline queue relative to the chosen direction: if the deadline of the first request in the queue is elapsed, the elevator moves that request to the tail of the dispatch queue; it also moves a batch of requests taken from the sorted queue, starting from the request following the expired one. The length of this batch is longer if the requests happen to be physically adjacent on disks, shorter otherwise.

Finally, if no request is expired, the elevator dispatches a batch of requests starting with the request following the last one taken from the sorted queue. When the cursor reaches the tail of the sorted queue, the search starts again from the top ("one-way elevator").

14.3.4.4. The "Anticipatory" elevator

The "Anticipatory" elevator is the most sophisticated I/O scheduler algorithm offered by Linux. Basically, it is an evolution of the "Deadline" elevator, from which it borrows the fundamental mechanism: there are two deadline queues and two sorted queues; the I/O scheduler keeps scanning the sorted queues, alternating between read and write requests, but giving preference to the read ones. The scanning is basically sequential, unless a request expires. The default expire time for read requests is 125 milliseconds, while the default expire time for write requests is 250 milliseconds. The elevator, however, follows some additional heuristics:

- In some cases, the elevator might choose a request behind the current position in the sorted queue, thus forcing a backward seek of the disk head. This happens, typically, when the seek distance for the request behind is less than half the seek distance of the request after the current position in the sorted queue.
- The elevator collects statistics about the patterns of I/O operations triggered by every process in the system. Right after dispatching a read request that comes from some process P, the elevator checks whether the next request in the sorted queue comes from the same process P. If so, the next request is dispatched immediately. Otherwise, the elevator looks at the collected statistics about process P: if it decides that process P will likely issue another read request soon, then it stalls for a short period of time (by default, roughly 7 milliseconds). Thus, the elevator might anticipate a read request coming from process P that is "close" on disk to the request just dispatched.

14.3.5. Issuing a Request to the I/O Scheduler

As seen in the section "[Submitting a Request](#)" earlier in this chapter, the `generic_make_request()` function invokes the `make_request_fn` method of the request queue descriptor to transmit a request to the I/O scheduler. This method is usually implemented by the `__make_request()` function; it receives as its parameters a `request_queue` descriptor `q` and a `bio` descriptor `bio`, and it performs the following operations:

1. Invokes the `blk_queue_bounce()` function to set up a bounce buffer, if required (see later). If a bounce buffer was created, the `__make_request()` function operates on it rather than on the original `bio`.
2. Invokes the I/O scheduler function `elv_queue_empty()` to check whether there are pending requests in the request queue; notice that the dispatch queue might be empty, but other queues of the I/O scheduler might contain pending requests. If there are no pending requests, it invokes the `blk_plug_device()` function to plug the request queue (see the section "[Activating the Block Device Driver](#)" earlier in this chapter), and jumps to step 5.
3. Here the request queue includes pending requests. Invokes the `elv_merge()` I/O scheduler function to check whether the new `bio` can be merged inside an existing request. The function may return three possible values:
 - ◆ `ELEVATOR_NO_MERGE`: the `bio` cannot be included in an already existing request: in that case, the function jumps to step 5.
 - ◆ `ELEVATOR_BACK_MERGE`: the `bio` might be added as the last `bio` of some request `req`: in that case, the function invokes the `q->back_merge_fn` method to check whether the request can be extended. If not, the function jumps to step 5. Otherwise it inserts the `bio` descriptor at the tail of the `req`'s list and updates the `req`'s fields. Then, it tries to merge the request with a following request (the new `bio` might fill a hole between the two requests).
 - ◆ `ELEVATOR_FRONT_MERGE`: the `bio` can be added as the first `bio` of some request `req`: in that case, the function invokes the `q->front_merge_fn` method to check whether the request can be extended. If not, it jumps to step 5. Otherwise, it inserts the `bio` descriptor at the head of the `req`'s list and updates the `req`'s fields. Then, the function tries to merge the request with the preceding request.
4. The `bio` has been merged inside an already existing request. Jumps to step 7 to terminate the function.
5. Here the `bio` must be inserted in a new request. Allocates a new request descriptor. If there is no free memory, the function suspends the current process, unless the `BIO_RW_AHEAD` flag in `bio->bi_rw` is set, which means that the I/O operation is a read-ahead (see [Chapter 16](#)); in this case, the function invokes `bio_endio()` and terminates: the data transfer will not be executed. For a description of `bio_endio()`, see step 1 of `generic_make_request()` in the earlier section "[Submitting a Request](#)."
6. Initializes the fields of the request descriptor. In particular:
 - a. Initializes the various fields that store the sector numbers, the current `bio`, and the current segment according to the contents of the `bio` descriptor.
 - b. Sets the `REQ_CMD` flag in the `flags` field (this is a normal read or write operation).
 - c. If the page frame of the first `bio` segment is in low memory, it sets the `buffer` field to the linear address of that buffer.
 - d. Sets the `rq_disk` field with the `bio->bi_bdev->bd_disk` address.
 - e. Inserts the `bio` in the request list.
 - f. Sets the `start_time` field to the value of `jiffies`.
7. All done. Before terminating, however, it checks whether the `BIO_RW_SYNC` flag in `bio->bi_rw` is set. If so, it invokes `generic_unplug_device()` on the request queue to unplug the driver (see the section "[Activating the Block Device Driver](#)" earlier in this chapter).

8. Terminates.

If the request queue was not empty before invoking `__make_request()`, either the request queue is already unplugged, or it will be unplugged soon because each plugged request queue `q` with pending requests has a running `q->unplug_timer` dynamic timer. On the other hand, if the request queue was empty, the `__make_request()` function plugs it. Sooner (on exiting from `__make_request()`, if the `BIO_RW_SYNC` bio flag is set) or later (in the worst case, when the unplug timer decays), the request queue will be unplugged. In any case, eventually the strategy routine of the block device driver will take care of the requests in the dispatch queue (see the section "[Device Driver Registration and Initialization](#)" earlier in this chapter).

14.3.5.1. The `blk_queue_bounce()` function

The `blk_queue_bounce()` function looks at the flags in `q->bounce_gfp` and at the threshold in `q->bounce_pfn` to determine whether buffer bouncing might be required. This happens when some of the buffers in the request are located in high memory and the hardware device is not able to address them.

Older DMA for ISA buses only handled 24-bit physical addresses. In this case, the buffer bouncing threshold is set to 16 MB, that is, to page frame number 4096. Block device drivers, however, do not usually rely on buffer bouncing when dealing with older devices; rather, they prefer to directly allocate the DMA buffers in the `ZONE_DMA` memory zone.

If the hardware device cannot cope with buffers in high memory, the function checks whether some of the buffers in the bio must really be bounced. In this case, it makes a copy of the bio descriptor, thus creating a bounce bio; then, for each segment's page frame having number equal to or greater than `q->bounce_pfn`, it performs the following steps:

1. Allocates a page frame in the `ZONE_NORMAL` or `ZONE_DMA` memory zone, according to the allocation flags.
2. Updates the `bv_page` field of the segment in the bounce bio so that it points to the descriptor of the new page frame.
3. If `bio->bio_rw` specifies a write operation, it invokes `kmap()` to temporarily map the high memory page in the kernel address space, copies the high memory page onto the low memory page, and invokes `kunmap()` to release the mapping.

The `blk_queue_bounce()` function then sets the `BIO_BOUNCED` flag in the bounce bio, initializes a specific `bi_end_io` method for the bounce bio, and finally stores in the `bi_private` field of the bounce bio the pointer to the original bio. When the I/O data transfer on the bounce bio terminates, the function that implements the `bi_end_io` method copies the data to the high memory buffer (only for a read operation) and releases the bounce bio.

14.4. Block Device Drivers

Block device drivers are the lowest component of the Linux block subsystem. They get requests from I/O scheduler, and do whatever is required to process them.

Block device drivers are, of course, integrated within the device driver model described in the section "[The Device Driver Model](#)" in [Chapter 13](#). Therefore, each of them refers to a `device_driver` descriptor; moreover, each disk handled by the driver is associated with a `device` descriptor. These descriptors, however, are rather generic: the block I/O subsystem must store additional information for each block device in the system.

14.4.1. Block Devices

A block device driver may handle several block devices. For instance, the IDE device driver can handle several IDE disks, each of which is a separate block device. Furthermore, each disk is usually partitioned, and each partition can be seen as a logical block device. Clearly, the block device driver must take care of all VFS system calls issued on the block device files associated with the corresponding block devices.

Each block device is represented by a `block_device` descriptor, whose fields are listed in [Table 14-9](#).

Table 14-9. The fields of the block device descriptor

Type	Field	Description
<code>dev_t</code>	<code>bd_dev</code>	Major and minor numbers of the block device
<code>struct inode *</code>	<code>bd_inode</code>	Pointer to the inode of the file associated with the block device in the bdev filesystem
<code>int</code>	<code>bd_openers</code>	Counter of how many times the block device has been opened
<code>struct semaphore</code>	<code>bd_sem</code>	Semaphore protecting the opening and closing of the block device
<code>struct semaphore</code>	<code>bd_mount_sem</code>	Semaphore used to forbid new mounts on the block device
<code>struct list_head</code>	<code>bd_inodes</code>	Head of a list of inodes of opened block device files for this block

		device
<code>void *</code>	<code>bd_holder</code>	Current holder of block device descriptor
<code>int</code>	<code>bd_holders</code>	Counter for multiple settings of the <code>bd_holder</code> field
<code>struct</code>		If block device is a partition, it points to the block device descriptor of the whole disk;
<code>block_device *</code>	<code>bd_contains</code>	otherwise, it points to this block device descriptor
<code>unsigned</code>	<code>bd_block_size</code>	Block size
<code>struct hd_struct *</code>	<code>bd_part</code>	Pointer to partition descriptor (NULL if this block device is not a partition)
<code>unsigned</code>	<code>bd_part_count</code>	Counter of how many times partitions included in this block device have been opened
<code>int</code>	<code>bd_invalidated</code>	Flag set when the partition table on this block device needs to be read
<code>struct gendisk *</code>	<code>bd_disk</code>	Pointer to <code>gendisk</code> structure of the disk underlying this block device
<code>struct list_head *</code>	<code>bd_list</code>	Pointers for the block device descriptor list
<code>struct</code>	<code>bd_inode_back</code>	Pointer to a specialized <code>backing_dev_info</code> descriptor for this block device (usually NULL)
<code>backing_dev_info *</code>	<code>ing_dev_info</code>	
<code>unsigned long</code>	<code>bd_private</code>	Pointer to private data of the block device holder

All block device descriptors are inserted in a global list, whose head is represented by the `all_bdevs` variable; the pointers for list linkage are in the `bd_list` field of the block device descriptor.

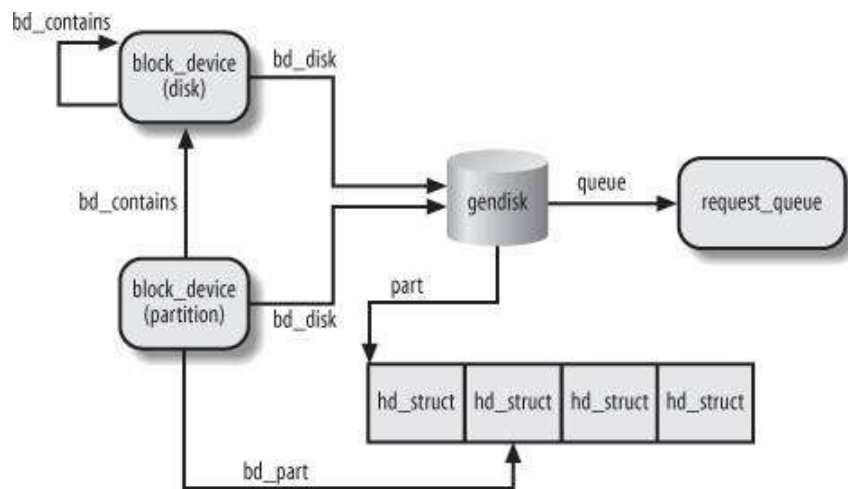
If the block device descriptor refers to a disk partition, the `bd_contains` field points to the descriptor of the block device associated with the whole disk, while the `bd_part` field points to the `hd_struct` partition descriptor (see the section "[Representing Disks and Disk Partitions](#)" earlier in this chapter). Otherwise, if the block device descriptor refers to a whole disk, the `bd_contains` field points to the block device descriptor itself, and the `bd_part_count` field records how many times the partitions on the disk have been opened.

The `bd_holder` field stores a linear address representing the holder of the block device. The holder is not the block device driver that services the I/O data transfers of the device; rather, it is a kernel component that makes use of the device and has exclusive, special privileges (for instance, it can freely use the `bd_private` field of the block device descriptor). Typically, the holder of a block device is the filesystem mounted over it. Another common case occurs when a block device file is opened for exclusive access: the holder is the corresponding file object.

The `bd_claim( )` function sets the `bd_holder` field with a specified address; conversely, the `bd_release( )` function resets the field to NULL. Be aware, however, that the same kernel component can invoke `bd_claim( )` many times; each invocation increases the `bd_holders` field. To release the block device, the kernel component must invoke `bd_release( )` a corresponding number of times.

Figure 14-3 refers to a whole disk and illustrates how the block device descriptors are linked to the other main data structures of the block I/O subsystem.

Figure 14-3. Linking the block device descriptors with the other structures of the block subsystem



14.4.1.1. Accessing a block device

When the kernel receives a request for opening a block device file, it must first determine whether the device file is already open. In fact, if the file is already open, the kernel must not create and initialize a new block device descriptor; rather, it should update the already existing block device descriptor. To complicate life, block device files that have the same major and minor numbers but different pathnames are viewed by the VFS as different files, although they really refer to the same block device. Therefore, the kernel cannot determine whether a block device is already in use by simply checking for the existence in the inode cache of an object for a block device file.

The relationship between a major and minor number and the corresponding block device descriptor is maintained through the bdev special filesystem (see the section "[Special Filesystems](#)" in [Chapter 12](#)). Each block device descriptor is coupled with a bdev special file: the `bd_inode` field of the block device descriptor points to the corresponding bdev inode; conversely, such an inode encodes both the major and minor numbers of the block device and the address of the corresponding descriptor.

The `bdget ( )` function receives as its parameter the major and minor numbers of a block device: It looks up in the bdev filesystem the associated inode; if such inode does not exist, the function allocates a new inode and new block device descriptor. In any case, the function returns the address of the block device descriptor corresponding to given major and minor numbers.

Once the block device descriptor for a block device has been found, the kernel can determine whether the block device is currently in use by checking the value of the `bd_openers` field: if it is positive, the block device is already in use (possibly by means of a different device file). The kernel also maintains a list of inode objects relative to opened block device files. This list is rooted at the `bd_inodes` field of the block device descriptor; the `i_devices` field of the inode object stores the pointers for the previous and next element in this list.

14.4.2. Device Driver Registration and Initialization

Let's now explain the essential steps involved in setting up a new device driver for a block device. Clearly, the description that follows is very succinct, nevertheless it could be useful to understand how and when the main data structures used by the block I/O subsystem are initialized.

We silently omit many steps required for all kinds of device drivers and already mentioned in [Chapter 13](#). For example, we skip all steps required for registering the driver itself (see the section "[The Device Driver Model](#)" in [Chapter 13](#)). Usually, the block device belongs to a standard bus architecture such as PCI or SCSI, and the kernel offers helper functions that, as a side effect, register the driver in the device driver model.

14.4.2.1. Defining a custom driver descriptor

First of all, the device driver needs a custom descriptor *foo* of type *foo_dev_t* holding the data required to drive the hardware device. For every device, the descriptor will store information such as the I/O ports used to program the device, the IRQ line of the interrupts raised by the device, the internal status of the device, and so on. The descriptor must also include a few fields required by the block I/O subsystem:

```
struct foo_dev_t {
    [...]
    spinlock_t lock;
    struct gendisk *gd;
    [...]
} foo;
```

The `lock` field is a spin lock used to protect the fields of the *foo* descriptor; its address is often passed to kernel helper functions, which can thus protect the data structures of the block I/O subsystem specific to the driver. The `gd` field is a pointer to the *gendisk* descriptor that represents the whole block device (disk) handled by this driver.

Reserving the major number

The device driver must reserve a major number for its own purposes. Traditionally, this is done by invoking the `register_blkdev( )` function:

```
err = register_blkdev(FOO_MAJOR, "foo");
if (err) goto error_major_is_busy;
```

This function is very similar to `register_chrdev( )` presented in the section "[Assigning Device Numbers](#)" in [Chapter 13](#): it reserves the major number *FOO_MAJOR* and associates the name *foo* to it. Notice that there is no way to allocate a subrange of minor numbers, because there is no analog of `register_chrdev_region( )`; moreover, no link is established between the reserved major number and the data structures of the driver. The only visible effect of `register_blkdev( )` is to include a new item in the list of registered major numbers in the */proc/devices* special file.

14.4.2.2. Initializing the custom descriptor

All the fields of the *foo* descriptor must be initialized properly before making use of the driver. To initialize the fields related to the block I/O subsystem, the device driver could execute the following instructions:

```
spin_lock_init(&foo.lock);
foo.gd = alloc_disk(16);
if (!foo.gd) goto error_no_gendisk;
```

The driver initializes the spin lock, then allocates the disk descriptor. As shown earlier in [Figure 14-3](#), the *gendisk* structure is crucial for the block I/O subsystem, because it references many other data structures. The `alloc_disk( )` function allocates also the array that stores the partition descriptors of the disk. The argument of the function is the number of *hd_struct* elements in the array; the value 16 means that the driver can support disks containing up to 15 partitions (partition 0 is not used).

14.4.2.3. Initializing the gendisk descriptor

Next, the driver initializes some fields of the *gendisk* descriptor:

```
foo.gd->private_data = &foo;
foo.gd->major = FOO_MAJOR;
foo.gd->first_minor = 0;
foo.gd->minors = 16;
set_capacity(foo.gd, foo_disk_capacity_in_sectors);
strcpy(foo.gd->disk_name, "foo");
foo.gd->fops = &foo_ops;
```

The address of the *foo* descriptor is saved in the *private_data* of the *gendisk* structure, so that low-level driver functions invoked as methods by the block I/O subsystem can quickly find the driver descriptor. This improves efficiency if the driver can handle more than one disk at a time. The `set_capacity( )` function initializes the *capacity* field with the size of the disk in 512-byte sectors. This value is likely determined by probing the hardware and asking about the disk parameters.

14.4.2.4. Initializing the table of block device methods

The *fops* field of the *gendisk* descriptor is initialized with the address of a custom table of block device methods (see [Table 14-4](#) earlier in this chapter).^[*] Quite likely, the *foo_ops* table of the device driver includes functions specific to the device driver. As an example, if the hardware device supports removable disks, the generic block layer may invoke the *media_changed* method to check whether the disk is changed since the last mount or open operation on the block device. This check is usually done by sending some low-level commands to the hardware controller, thus the implementation of the *media_changed* method is always specific to the device driver.

[*] The block device methods should not be confused with the block device file operations; see the section "[Opening a Block Device File](#)" later in this chapter.

Similarly, the *ioctl* method is only invoked when the generic block layer does not know how to handle some *ioctl* command. For instance, the method is typically invoked when an `ioctl( )` system call asks about the disk geometry, that is, the number of cylinders, tracks, sectors, and heads used by the disk. Thus, the implementation of this method is specific to the device driver.

14.4.2.5. Allocating and initializing a request queue

Our brave device driver designer might now set up a request queue that will collect the requests waiting to be serviced. This can be easily done as follows:

```
foo.gd->rq = blk_init_queue(foo_strategy, &foo.lock);
if (!foo.gd->rq) goto error_no_request_queue;
blk_queue_hardsect_size(foo.gd->rd, foo_hard_sector_size);
blk_queue_max_sectors(foo.gd->rd, foo_max_sectors);
blk_queue_max_hw_segments(foo.gd->rd, foo_max_hw_segments);
blk_queue_max_phys_segments(foo.gd->rd, foo_max_phys_segments);
```

The `blk_init_queue( )` function allocates a request queue descriptor and initializes many of its fields with default values. It receives as its parameters the address of the device descriptor's spin lock for the `foo.gd->rq->queue_lock` field and the address of the strategy routine of the device driver for the `foo.gd->rq->request_fn` field; see the next section; "[The Strategy Routine](#)." The `blk_init_queue( )` function also initializes the `foo.gd->rq->elevator` field, forcing the driver to use the default I/O scheduler algorithm. If the device driver wants to use a different elevator, it may later override the address in the `elevator` field.

Next, some helper functions set various fields of the request queue descriptor with the proper values for the device driver (look at [Table 14-6](#) for the similarly named fields).

14.4.2.6. Setting up the interrupt handler

As described in the section "[I/O Interrupt Handling](#)" in [Chapter 4](#), the driver needs to register the IRQ line for the device. This can be done as follows:

```
request_irq(foo_irq, foo_interrupt,
           SA_INTERRUPT|SA_SHIRQ, "foo", NULL);
```

The `foo_interrupt( )` function is the interrupt handler for the device; we discuss some of its peculiarities in the section "[The Interrupt Handler](#)" later in this chapter).

14.4.2.7. Registering the disk

Finally, all the device driver's data structures are ready: the last step of the initialization phase consists of "registering" and activating the disk. This can be achieved simply by executing:

```
add_disk(foo.gd);
```

The `add_disk( )` function receives as its parameter the address of the `gendisk` descriptor, and essentially executes the following operations:

1. Sets the `GENHD_FL_UP` flag of `gd->flags`.
2. Invokes `kobj_map( )` to establish the link between the device driver and the device's major number with its associated range of minor numbers (see the section "[Character Device Drivers](#)" in [Chapter 13](#);

- be warned that in this case the kobject mapping domain is represented by the `bdev_map` variable).
3. Registers the kobject included in the `gendisk` descriptor in the device driver model as a new device serviced by the device driver (e.g., `/sys/block/foo`).
 4. Scans the partition table included in the disk, if any; for each partition found, properly initializes the corresponding `hd_struct` descriptor in the `foo.gd->part` array. Also registers the partitions in the device driver model (e.g., `/sys/block/foo/foo1`).
 5. Registers the kobject embedded in the request queue descriptor in the device driver model (e.g., `/sys/block/foo/queue`).

Once `add_disk()` returns, the device driver is actively working. The function that carried on the initialization phase terminates; the strategy routine and the interrupt handler take care of each request passed to the device driver by the I/O scheduler.

14.4.3. The Strategy Routine

The strategy routine is a function or a group of functions of the block device driver that interacts with the hardware block device to satisfy the requests collected in the dispatch queue. The strategy routine is invoked by means of the `request_fn` method of the request queue descriptor the `foo_strategy()` function in the example carried on in the previous section. The I/O scheduler layer passes to this function the address `q` of the request queue descriptor.

As we'll see, the strategy routine is usually started after inserting a new request in an empty request queue. Once activated, the block device driver should handle all requests in the queue and terminate when the queue is empty.

A naïve implementation of the strategy routine could be the following: for each element in the dispatch queue, remove it from the queue, interact with the block device controller to service the request, and wait until the data transfer completes. Then proceed with the next request in the dispatch queue.

Such an implementation is not very efficient. Even assuming that the data can be transferred using DMA, the strategy routine must suspend itself while waiting for I/O completion. This means that the strategy routine should execute on a dedicated kernel thread (we don't want to penalize an unrelated user process, do we?). Moreover, such a driver would not be able to support modern disk controllers that can process multiple I/O data transfers at a time.

Therefore, most block device drivers adopt the following strategy:

- The strategy routine starts a data transfer for the first request in the queue and sets up the block device controller so that it raises an interrupt when the data transfer completes. Then the strategy routine terminates.
- When the disk controller raises the interrupt, the interrupt handler invokes the strategy routine again (often directly, sometimes by activating a work queue). The strategy routine either starts another data transfer for the current request or, if all the chunks of data of the request have been transferred, removes the request from the dispatch queue and starts processing the next request.

Requests can be composed of several bios, which in turn can be composed of several segments. Basically, block device drivers make use of DMA in two ways:

- The driver sets up a different DMA transfer to service each segment in each bio of the request
- The driver sets up a single scatter-gather DMA transfer to service all segments in all bios of the request

Ultimately, the design of the strategy routine of the device drivers depends on the characteristics of the block controller. Each physical block device is inherently different from all others (for example, a floppy driver groups blocks in disk tracks and transfers a whole track in a single I/O operation), so making general assumptions on how a device driver should service a request is meaningless.

In our example, the `foo_strategy( )` strategy routine could execute the following actions:

1. Gets the current request from the dispatch queue by invoking the `elv_next_request( )` I/O scheduler helper function. If the dispatch queue is empty, the strategy routine returns:

```
req = elv_next_request(q);
if (!req) return;
```

2. Executes the `blk_fs_request` macro to check whether the `REQ_CMD` flag of the request is set, that is, whether the request contains a normal read or write operation:

```
if (!blk_fs_request(req))
    goto handle_special_request;
```

3. If the block device controller supports scatter-gather DMA, it programs the disk controller so as to perform the data transfer for the whole request and to raise an interrupt when the transfer completes. The `blk_rq_map_sg( )` helper function returns a scatter-gather list that can be immediately used to start the transfer.
4. Otherwise, the device driver must transfer the data segment by segment. In this case, the strategy routine executes the `rq_for_each_bio` and `bio_for_each_segment` macros, which walk the list of bios and the list of segments inside each bio, respectively:

```
rq_for_each_bio(bio, rq)
    bio_for_each_segment(bvec, bio, i) {
        /* transfer the i-th segment bvec */
        local_irq_save(flags);
        addr = kmap_atomic(bvec->bv_page, KM_BIO_SRC_IRQ);
        foo_start_dma_transfer(addr+bvec->bv_offset, bvec->bv_len);
        kunmap_atomic(bvec->bv_page, KM_BIO_SRC_IRQ);
        local_irq_restore(flags);
    }
```

The `kmap_atomic( )` and `kunmap_atomic( )` functions are required if the data to be transferred can be in high memory. The `foo_start_dma_transfer( )` function programs the hardware device so as to start the DMA transfer and to raise an interrupt when the I/O operation completes.

5. Returns.

14.4.4. The Interrupt Handler

The interrupt handler of a block device driver is activated when a DMA transfer terminates. It should check whether all chunks of data in the request have been transferred. If so, the interrupt handler invokes the strategy routine to process the next request in the dispatch queue. Otherwise, the interrupt handler updates the field of the request descriptor and invokes the strategy routine to process the data transfer yet to be performed.

A typical fragment of the interrupt handler of our *foo* device driver is the following:

```
irqreturn_t foo_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
    struct foo_dev_t *p = (struct foo_dev_t *) dev_id;
    struct request_queue *rq = p->gd->rq;
    [...]
    if (!end_that_request_first(rq, uptodate, nr_sectors)) {
        blkdev_dequeue_request(rq);
        end_that_request_last(rq);
    }
    rq->request_fn(rq);
    [...]
    return IRQ_HANDLED;
}
```

The job of ending a request is split in two functions called `end_that_request_first( )` and `end_that_request_last( )`.

The `end_that_request_first( )` function receives as arguments a request descriptor, a flag indicating if the DMA data transfer completed successfully, and the number of sectors transferred in the DMA transfer (the `end_that_request_chunk( )` function is similar, but it receives the number of bytes transferred instead of the number of sectors). Essentially, the function scans the bios in the request and the segments inside each bio, then updates the fields of the request descriptor in such a way to:

- Set the `bio` field so that it points to the first unfinished bio in the request.
- Set the `bi_idx` of the unfinished bio so that it points to the first unfinished segment.
- Set the `bv_offset` and `bv_len` fields of the unfinished segment so that they specify the data yet to be transferred.

The function also invokes `bio_endio( )` on each bio that has been completely transferred.

The `end_that_request_first( )` function returns 0 if all chunks of data in the request have been transferred; otherwise, it returns 1. If the returned value is 1, the interrupt handler restarts the strategy routine, which thus continues processing the same request. Otherwise, the interrupt handler removes the request from the request queue (typically by using `blkdev_dequeue_request( )`), invokes the `end_that_request_last( )` helper function, and restarts the strategy routine to process the next request in the dispatch queue.

The `end_that_request_last( )` function updates some disk usage statistics, removes the request descriptor from the dispatch queue of the `rq->elevator` I/O scheduler, wakes up any process sleeping in the waiting completion of the request descriptor, and releases that descriptor.

14.5. Opening a Block Device File

We conclude this chapter by describing the steps performed by the VFS when opening a block device file.

The kernel opens a block device file every time that a filesystem is mounted over a disk or partition, every time that a swap partition is activated, and every time that a User Mode process issues an `open ( )` system call on a block device file. In all cases, the kernel executes essentially the same operations: it looks for the block device descriptor (possibly allocating a new descriptor if the block device is not already in use), and sets up the file operation methods for the forthcoming data transfers.

In the section "[VFS Handling of Device Files](#)" in [Chapter 13](#), we described how the `dentry_open ( )` function customizes the methods of the file object when a device file is opened. In this case, the `f_op` field of the file object is set to the address of the `def_blk_fops` table, whose content is shown in [Table 14-10](#).

Table 14-10. The default block device file operations (def_blk_fops table)

Method	Function
<code>open</code>	<code>blkdev_open ()</code>
<code>release</code>	<code>blkdev_close ()</code>
<code>llseek</code>	<code>block_llseek ()</code>
<code>read</code>	<code>generic_file_read ()</code>
<code>write</code>	<code>blkdev_file_write ()</code>
<code>aio_read</code>	<code>generic_file_aio_read ()</code>
<code>aio_write</code>	<code>blkdev_file_aio_write ()</code>
<code>mmap</code>	<code>generic_file_mmap ()</code>
<code>fsync</code>	<code>block_fsync ()</code>
<code>ioctl</code>	<code>block_ioctl ()</code>
<code>compat_ioctl</code>	<code>compat_blkdev_ioctl ()</code>
<code>readv</code>	<code>generic_file_readv ()</code>
<code>writev</code>	<code>generic_file_write_nolock ()</code>
<code>sendfile</code>	<code>generic_file_sendfile ()</code>

Here we are only concerned with the `open` method, which is invoked by the `dentry_open ( )` function. The `blkdev_open ( )` function receives as its parameters `inode` and `filp`, which store the addresses of the inode and file objects respectively; the function essentially executes the following steps:

1. Executes `bd_acquire (inode )` to get the address `bdev` of the block device descriptor. In turn, this function receives the inode object address and performs the following steps:
 - a. Checks whether the `inode->i_bdev` field of the inode object is not `NULL`; if it is, the block device file has been opened already, and this field stores the address of the corresponding block descriptor. In this case, the function increases the usage counter of the

- inode->i_bdev->bd_inode inode of the bdev special filesystem associated with the block device, and returns the address inode->i_bdev of the descriptor.
 - b. Here the block device file has not been opened yet. Executes `bdget (inode->i_rdev)` to get the address of the block device descriptor corresponding to the major and minor number of the block device file (see the section "[Block Devices](#)" earlier in this chapter). If the descriptor does not already exist, `bdget ( )` allocates it; notice however that the descriptor might already exist, for instance because the block device is already being accessed by means of another block device file.
 - c. Stores the block device descriptor address in `inode->i_bdev`, so as to speed up future opening operations on the same block device file.
 - d. Sets the `inode->i_mapping` field with the value of the corresponding field in the bdev inode. This is the pointer to the address space object, which will be explained in the section "[The address_space Object](#)" in [Chapter 15](#).
 - e. Inserts `inode` into the list of opened inodes of the block device descriptor rooted at `bdev->bd_inodes`.
 - f. Returns the address `bdev` of the descriptor.
2. Sets the `filp->i_mapping` field with the value of `inode->i_mapping` (see step 1(d) above).
 3. Gets the address of the `gendisk` descriptor relative to this block device:

```
disk = get_gendisk(bdev->bd_dev, &part);
```

If the block device being opened is a partition, the function also returns its index in the `part` local variable; otherwise, `part` is set to zero. The `get_gendisk ( )` function simply invokes `kobj_lookup ( )` on the `bdev_map` kobject mapping domain passing the major and minor number of the device (see also the section "[Device Driver Registration and Initialization](#)" earlier in this chapter).

4. If `bdev->bd_openers` is not equal to zero, the block device has already been opened. Checks the `bdev->bd_contains` field:
 - a. If it is equal to `bdev`, the block device is a whole disk: invokes the `bdev->bd_disk->fops->open` block device method, if defined, then checks the `bdev->bd_invalidated` field and invokes, if necessary, the `rescan_partitions ( )` functions (see comments on steps 6a and 6c later).
 - b. If it not equal to `bdev`, the block device is a partition: increases the `bdev->bd_contains->bd_part_count` counter.

Then, jumps to step 8.

5. Here the block device is being accessed for the first time. Initializes `bdev->bd_disk` with the address `disk` of the `gendisk` descriptor.
6. If the block device is a whole disk (`part` is zero), it executes the following substeps:
 - a. If defined, it executes the `disk->fops->open` block device method: it is a custom function defined by the block device driver to perform any specific last-minute initialization.
 - b. Gets from the `hardsect_size` field of the `disk->queue` request queue the sector size in bytes, and uses this value to set properly the `bdev->bd_block_size` and `bdev->bd_inode->i_blkbits` fields. Sets also the `bdev->bd_inode->i_size` field with the size of the disk computed from `disk->capacity`.
 - c. If the `bdev->bd_invalidated` flag is set, it invokes `rescan_partitions ( )` to scan the partition table and update the partition descriptors. The flag is set by the `check_disk_change` block device method, which applies only to removable devices.
7. Otherwise if the block device is a partition (`part` is not zero), it executes the following substeps:
 - a. Invokes `bdget ( )` again this time passing the `disk->first_minor` minor number to get the address `whole` of the block descriptor for the whole disk.

- b. Repeats steps from 3 to 6 for the block device descriptor of the whole disk, thus initializing it if necessary.
- c. Sets `bdev->bd_contains` to the address of the descriptor of the whole disk.
- d. Increases `whole->bd_part_count` to account for the new open operation on the partition of the disk.
- e. Sets `bdev->bd_part` with the value in `disk->part[part-1]`; it is the address of the `hd_struct` descriptor of the partition. Also, executes `kobject_get(&bdev->bd_part->kobj)` to increase the reference counter of the partition.
- f. As in step 6b, sets the inode fields that specify size and sector size of the partition.
- 8. Increases the `bdev->bd_openers` counter.
- 9. If the block device file is being opened in exclusive mode (`O_EXCL` flag in `filp->f_flags` set), it invokes `bd_claim(bdev, filp)` to set the holder of the block device (see the section "[Block Devices](#)" earlier in this chapter). In case of error block device has already an holder it releases the block device descriptor and returns the error code `-EBUSY`.
- 10. Terminates by returning 0 (success).

Once the `blkdev_open()` function terminates, the `open()` system call proceeds as usual. Every future system call issued on the opened file will trigger one of the default block device file operations. As we will see in [Chapter 16](#), each data transfer to or from the block device is effectively implemented by submitting requests to the generic block layer.